第 2 章 进程与线程管理

进

2.1 概述

2.1.1 StarryOS

StarryOS 进 `Thread` `ProcessData`

StarryOS `fork` `clone` `clone3` `execve` `exit` `waitpid` 进 进 pidfd进 moss Linux

2.1.2 base os

“ 进 ” 进 StarryOS

`clone3` pidfd `execve` `execveat` `MS_NOEXEC`

`waitpid` 进 ” “ ” `waitid` `siginfo_t` UID/GID capability 进 `kill` ptrace pidfd `waitid` `/proc` 进 进

2.1.3 moss

moss StarryOS

	StarryOS	moss
	`Thread` `ProcessData`	/ namespace
	`clone3` `execve`	clone ABI `execveat`
	`waitpid`	进 `waitid` pidfd
	进 pidfd	ptrace

2-1 进

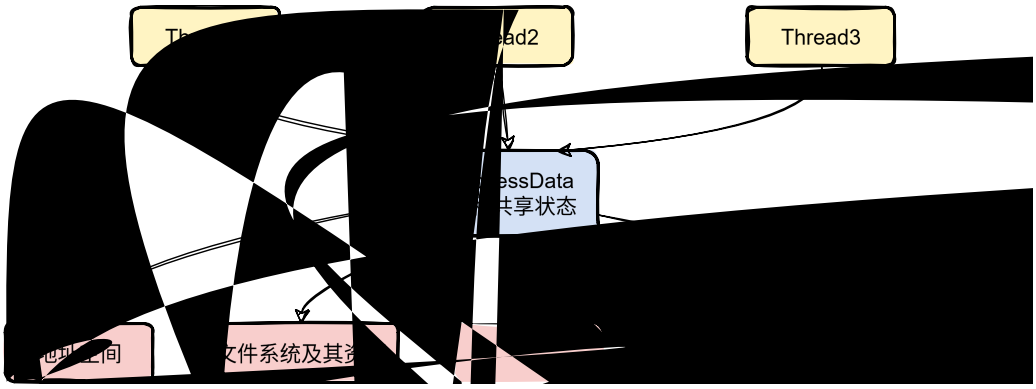
2.2 核心设计思想

StarryOS 进 moss 进
进 进

2.2.1

进 moss StarryOS
`Thread` `ProcessData` 进
`ProcessData` clone 进
进

`Thread`
`ProcessData` clone exec exit





exec PID 进
exit wait moss

2.3 关键数据结构

2.3.1 Thread ProcessData

Thread ProcessData

```rust
pub struct Thread {
    // 共享进程级资源
    pub proc_data: Arc<ProcessData>,
    // 线程私有状态
    pub signal: Arc<ThreadSignalManager>,
    pub exit: Arc<AtomicBool>,
}

pub struct ProcessData {
    // 进程身份及组织关系
    pub proc: Arc<Process>,
    // 线程共享的进程资源
    pub aspace: Arc<Mutex<AddrSpace>>,
    pub scope: RwLock<Scope>,
    pub wait_status: SpinNoIrq<Option<ProcessWaitStatus>>,
    uid: AtomicU32,
    euid: AtomicU32,
    suid: AtomicU32,
}
```

2-1 进
进 Arc<ProcessData>
Thread moss 进 进

Process ProcessData

“ ” “ 进 进 ” waitpid waitid

2.3.3

TID PID PGID SID moss

进

TASK_TABLE	TID	
PROCESS_TABLE	PID	进
PROCESS_GROUP_TABLE	PGID	进
SESSION_TABLE	SID	进

2-2 进

moss 进

PGID SID

2.4 进程与线程创建

2.4.1 fork clone

fork moss x86_64 sys_fork sys_clone

SIGCHLD 进 fork ProcessData

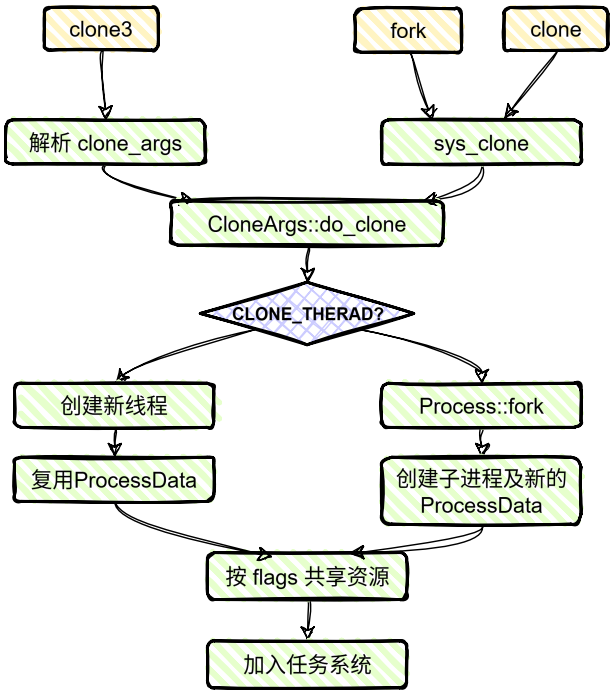
clone clone 进 进

CloneArgs CloneArgs::do_clone

CLONE_THREAD ProcessData Process::fork 进

ProcessData

CLONE_VM CLONE_FILES CLONE_FS CLONE_SIGHAND



2-2 fork clone clone3

CLONE_THREAD	ProcessData
CLONE_VM	
CLONE_FILES	
CLONE_FS	
CLONE_SIGHAND	

2-3 进 clone

CLONE_THREAD

2.4.2

moss clone

```
// 线程必须共享地址空间和信号处理动作
if flags.contains(CloneFlags::THREAD)
    && !flags.contains(CloneFlags::VM | CloneFlags::SIGHAND)
{
    return Err(AxError::InvalidInput);
}
// 共享信号处理动作必须共享地址空间
if flags.contains(CloneFlags::SIGHAND)
    && !flags.contains(CloneFlags::VM)
{
    return Err(AxError::InvalidInput);
}
```

2-3 clone

clone3 moss 进

2.4.3

进 进 pidfd

pidfd

moss

进 “ PID ”

CLONE_VFORK namespace 进

2.5 程序执行

2.5.1 execve

StarryOS execve ELF

CLOEXEC exec 进

PID 进

MS_NOEXEC

2.5.2 execveat

execveat dirfd

AT_EMPTY_PATH

AT_SYMLINK_NOFOLLOW

execve 进

execveat AT_EMPTY_PATH

Linux execve

moss execve execveat execve_common

moss

```
// 挂载点禁止执行
if exec_loc.mount_flags() & MS_NOEXEC != 0 {
    return Err(AxError::PermissionDenied);
}
// 按有效 UID/GID 检查执行位
if !has_exec_permission(exec_loc, proc_data.euid(), proc_data.egid())? {
    return Err(AxError::PermissionDenied);
}
// 拒绝正在被写入的可执行文件
if is_text_busy(exec_loc) {
    return Err(AxError::from(LinuxError::ETXTBSY));
}
```

2-4

2.5.3 exec

Linux 进 exec

moss exec 进

2.6 退出与资源回收

2.6.1

进 进

robust futex clear_child_tid 进

ProcessData

进 进 进

进 moss 进

ptrace 进 `wait` 进

2.6.2 exit_group

2.6.1 `exit_group` 进 进

moss

2.6.1 进

`exit` `exit_group` 进

2.6.3

进 进 进 进

StarryOS `children` 进 init

进

→ 进 init →进 进

进 PID 进 进

init 进 `wait`

2.7 子进程状态与等待

2.7.1

StarryOS `waitpid` moss 进

进 进 进

`WUNTRACED` `WCONTINUED` `WNOHANG` `waitpid` `wait4` `options`

`WUNTRACED`	进
`WCONTINUED`	`SIGCONT` 进
`WNOHANG`	0 进

2-4 waitpid wait4 options

`waitid` `WSTOPPED` `WEXITED` `WCONTINUED` `WNOWAIT`

moss `WSTOPPED` `WUNTRACED`

2.7.2 waitpid

`waitpid` pid 进 进 进

moss `Any` `Pid` `Pgid` 进 进

`ECHILD` 进 进

```rust
let target = match pid {
    -1 => WaitPid::Any, // 任意直接子进程
    0 => WaitPid::Pgid(proc.group().pgid()),
    1.. => WaitPid::Pid(pid as _),
    _ => WaitPid::Pgid(pid.checked_neg()? as _),
};

// waitpid 只能等待调用进程的直接子进程
let children = proc.children().into_iter()
    .filter(|child| target.apply(child))
    .collect::<Vec<_>>();
if children.is_empty() {
    return Err(LinuxError::ECHILD.into());
}
```

2-5 waitpid

2.7.3 waitid pidfd

moss waitid P_ALL P_PID P_PGID P_PIDFD
进

```rust
let (children, pidfd_nonblocking) = match idtype as u32 {
    // 全部直接子进程
    P_ALL => (proc.children(), false),
    // 按 PID 筛选
    P_PID => (proc.children().into_iter()
        .filter(|child| child.pid() == id).collect(), false),
    P_PIDFD => {
        // 由 pidfd 取得稳定进程引用
        let pidfd = PidFd::from_fd(id as _)?;
        let child_pid = pidfd.process_data()?.proc.pid();
        (proc.children().into_iter()
            .filter(|child| child.pid() == child_pid).collect(),
         pidfd.nonblocking())
    }
    _ => return Err(AxError::InvalidInput),
};
```

2-6 waitid

CLD_EXITED
CLD_KILLED CLD_STOPPED siginfo_t WNOWAIT 进
pidfd StarryOS moss waitid(P_PIDFD) /proc fdinfo
PID pidfd PID
进

2.8 进程组与会话

2.8.1 PID PGID SID

进 PID进 进 PGID 进 进
SID PID PGID PID SID进
Shell 进 进
进 进
moss StarryOS `Process` PID `ProcessGroup`
`ProcessGroup` PGID `Session` `Session` 进SID 进

```rust
pub struct Process {
    pid: Pid,
    // 每个进程只属于一个进程组
    group: SpinNoIrq<Arc<ProcessGroup>>,
}

pub struct ProcessGroup {
    pgid: Pid,
    // 进程组归属于一个会话
    session: Arc<Session>,
    // 记录组内的所有进程
    processes: SpinNoIrq<WeakMap<Pid, Weak<Process>>>,
}

pub struct Session {
    sid: Pid,
    // 一个会话可以包含多个进程组
    process_groups: SpinNoIrq<WeakMap<Pid, Weak<ProcessGroup>>>,
}
```

2-7 进 进
进

```rust
// 先按 PID 找到进程对象
let proc = get_process_data(pid)?.proc;
let pid  = proc.pid();
// 沿所属关系取得进程组和会话标识
let pgid = proc.group().pgid();
let sid  = proc.group().session().sid();
```

2-8 PID PGID SID

进
`ProcessGroup::processes` moss 进 `PROCESS_TABLE`
`PROCESS_GROUP_TABLE` `SESSION_TABLE` PID PGID SID

2.8.2

StarryOS `setpgid` `getpgid` 进 PGID
进 PGID 进

moss PGID PGID PID 进
进 `getpgid` 进

PGID 进 进
进
进
进 PGID

2.8.3

`setsid` moss 进 `Session` `ProcessGroup`

```
if let Some((session, group)) = proc.create_session() {
    // setsid 同时创建会话和进程组
    add_session_to_table(&session);
    add_process_group_to_table(&group);
    // SID 等于调用进程的 PID
    Ok(session.sid() as _)
}
```

2-9 进
PID PGID SID 进
进

2.9 权限与进程控制

2.9.1 UID/GID

Linux UID 进 real effective saved
real UID 进 effective UID
saved UID 进

```
pub struct ProcessData {
    uid: AtomicU32,  // real UID: 进程的实际身份
    euid: AtomicU32, // effective UID: 当前权限判断使用
    suid: AtomicU32, // saved UID: 保留可恢复的身份
}

let allowed = [old_ruid, old_euid, old_suid];
if old_euid != 0 {
    // 非特权进程不能把三元组改成从未拥有过的身份
    if !valid_id_change(ruid, &allowed)
        || !valid_id_change(euid, &allowed)
        || !valid_id_change(suid, &allowed)
    {
        return Err(eperm());
    }
}

if ruid != UID_GID_NO_CHANGE {
    proc_data.set_real_uid(ruid);
}
if euid != UID_GID_NO_CHANGE {
    proc_data.set_euid(euid);
}
if suid != UID_GID_NO_CHANGE {
    proc_data.set_suid(suid);
}
```

2-10 UID

进 effective UID 0 进

effective UID/GID `kill` ptrace attach

capability effective permitted inheritable bounding

进

2.9.2 ptrace

moss ptrace `TRACEME` `ATTACH` `DETACH` wait

attach

wait

`PTRACE_SYSCALL` `PTRACE_SINGLESTEP`

siginfo `EIO` GDB

2.9.3 namespace

moss PID UTS User namespace inode namespace init PID

clone unshare PID UID

namespace

2.10 本章小结

moss StarryOS 进 `Thread` `ProcessData` 进 real effective saved ID clone exec exit wait moss `clone3` ABI `execveat` pidfd ptrace vfork exec namespace ptrace 进 StarryOS